

Team Contest Reference

code with happiness, be confident in your work.

University of Twente – Team `(( => ))()`;

General Notes

- IO: `ios::sync_with_stdio(false); cin.tie(nullptr);` avoid `endl` in non-interactive (use `'\n'` instead).
- Types: prefer `int64_t` or `ll` for counts/weights, `ld` for geometry; define `EPS`.
- Compare floats with `EPS`: `(x>EPS)-(x<-EPS)`. Binary search on answer when the predicate is monotone (many no's $\rightarrow$ yes).
- Debug: assert invariants, test extremes, sanitize I/O format. Seed RNG for randomized algorithms.

1 Reasoning and Strategy

- Classify: implementation, DS, graphs, DP, math, geometry, strings, flows.
- Read constraints to pick $O(n)/O(n \log n)/O(n^2)$; check memory limits.
- Reformulate in primitives (graphs, intervals, sets). Identify invariant/monotonicity; attempt greedy proof or DP states.
- Solve core first, test trivial/edge. Time-box hard tasks; avoid sunk cost.
- Flow: 0–5m scan; 5–20m easy solves; 20–90m mediums; 90m+ reassess; leave buffer for debug.

General Notes

- IO: `ios::sync_with_stdio(false); cin.tie(nullptr);` avoid `endl` in non-interactive (use `'\n'` instead).
- Types: prefer `int64_t` or `ll` for counts/weights, `ld` for geometry; define `EPS`.
- Compare floats with `EPS`: `(x>EPS)-(x<-EPS)`. Binary search on answer when the predicate is monotone (many no's $\rightarrow$ yes).
- Debug: assert invariants, test extremes, sanitize I/O format. Seed RNG for randomized algorithms.

2 Reasoning and Strategy

- Classify: implementation, DS, graphs, DP, math, geometry, strings, flows.
- Read constraints to pick $O(n)/O(n \log n)/O(n^2)$; check memory limits.
- Reformulate in primitives (graphs, intervals, sets). Identify invariant/monotonicity; attempt greedy proof or DP states.
- Solve core first, test trivial/edge. Time-box hard tasks; avoid sunk cost.
- Flow: 0–5m scan; 5–20m easy solves; 20–90m mediums; 90m+ reassess; leave buffer for debug.

3 Function Reference

Math (`<cmath>`)

- `abs(x)`, `fabs(x)`: Absolute value (`int`, `double`).
- `pow(x, y)`, `sqrt(x)`, `cbirt(x)`, `hypot(x, y)`. - `sin(x)`
- `cos(x)`, `tan(x)`, `asin(x)`, `acos(x)`, `atan(x)`, `atan2(y, x)`
- `log(x)`, `log10(x)`, `exp(x)`. - `floor(x)`, `ceil(x)`, `round(x)`
- `trunc(x)`. - `fmod(x, y)`: Remainder. - `min(x, y)`, `max(x, y)`.

Algorithms (`<algorithm>`, `<numeric>`)

- `sort(begin, end)`, `sort(begin, end, cmp)`, `stable_sort`.
- `reverse(begin, end)`. - `next_permutation`, `prev_permutation`.
- `lower_bound`, `upper_bound`, `binary_search`. - `find`, `count`. - `min_element`, `max_element`.
- `accumulate(begin, end, init)` (`<numeric>`).

`adjacent_find`, `unique`. - `fill`, `copy`. - `partition`, `stable_partition`. - `rotate`, `nth_element`.

Vectors and Deques (`<vector>`, `<deque>`)

- `push_back`, `pop_back`, `insert`, `erase`, `resize`, `clear`, `assign`. - `front`, `back`, `at`, `operator[]`.

Associative Containers (`<set>`, `<map>`, `<unordered_set>`, `<unordered_map>`)

- `insert`, `erase`, `find`, `count`. - `lower_bound`, `upper_bound` (ordered only). - `begin`, `end`.

String (`<string>`)

- `length`, `size`, `empty`, `clear`. - `push_back`, `pop_back`, `append`. - `substr`, `find`, `rfind`, `find_first_of`, `find_last_of`. - `replace`, `insert`, `erase`. - `compare`, `c_str`.
- `stoi`, `stol`, `stoll`, `stoul`, `stoull`, `stof`, `stod`, `stold`: string $\rightarrow$ int/float. - `to_string`: int/float $\rightarrow$ string.

Pairs and Utility (`<utility>`)

- `pair<T1, T2>`, `make_pair`, `swap`, `tie`.

Standard Data Structures (`<queue>`, `<priority_queue>`, `<stack>`)

- `push`, `pop`, `front`, `back`, `top`, `empty`, `size`. - `priority_queue<T, Container, Compare>` (max/min heap).

Other Useful Builtins

- `__builtin_popcount(x)`: Number of set bits (GCC/Clang).
- `__builtin_ctz(x)`: Trailing zeros. - `__builtin_clz(x)`: Leading zeros.

Quick Examples

```
1 // Math
2 int r = abs(-5); // 5
3 double y = pow(2.0, 3.0); // 8.0
4
5 // Search
6 auto it = lower_bound(v.begin(), v.end(), 5);
7 if (binary_search(v.begin(), v.end(), 5)) { /* found */ }
8
9 // String to int
10 int x = stoi("123");
11 long long y = stoll("1234567890123");
12
13 // Int to string
14 std::string s = to_string(2025);
15
16 // Permutations
17 sort(v.begin(), v.end());
18 do { /* process permutation v */ } while (next_permutation(v.begin(), v.end()));
```

4 Geometry

4.1 Core Types and Helpers

```
1 using ld = long double;
2 const ld PI = acos(-1.0L), EPS = 1e-12L;
3 int sgn(ld x){ return (x>EPS)-(x<-EPS); } // sign of f.p. x
4 ld deg2rad(ld d){ return d*PI/180.0L; } // deg -> rad
5 ld rad2deg(ld r){ return r*180.0L/PI; } // rad -> deg
6
7 struct P{
8     ld x,y; // x,y of a point P
9     P(){ P(ld x,ld y):x(x),y(y){}
10    P operator+(P o) const { return {x+o.x,y+o.y}; } // vector add
11    P operator-(P o) const { return {x-o.x,y-o.y}; } // vector sub
12    P operator*(ld k) const { return {x*k,y*k}; } // scalar mul
13    P operator/(ld k) const { return {x/k,y/k}; } // scalar div
14 };
15 ld dot(P a,P b){ return a.x*b.x+a.y*b.y; } // dot prod
16 ld cross(P a,P b){ return a.x*b.y-a.y*b.x; } // cross prod
17 ld cross(P o,P a,P b){ return cross(a-o,b-o); } // orient area
18 ld norm2(P a){ return dot(a,a); } // squared norm
19 ld norm(P a){ return sqrtl(norm2(a)); } // euclid dist
```

4.2 Circles and Trig

- Circle: area $A = \pi r^2$; circ $C = 2\pi r$. sector area $\frac{1}{2}r^2\theta$ (rad); arc $s = R\theta$.